Smartphone Augmented Reality

Rapport écrit du Lab

GEORGI Thomas
thomas.georgi@imt-atlantique.net

KI Alidou
alidou.ki@imt-atlantique.net

IMT Atlantique
UE RMA - Semestre printemps 2026

Contents

1 Introduction

2 Création des éléments de jeu

2.1 La tourelle (Tower) . . . . . .

2.2 Les ennemis (Enemies) . . . . . .

2.3 Le gestionnaire de jeu (GameManager) . . . . . .

3 Techniques et Choix de Conception

3.1 Système de Coordonnées AR . . . . . .

3.2 Détection et Ciblage des Ennemis . . . . . .

3.3 Mouvement des ennemis . . . . . .

4 Problèmes Rencontrés et Solutions

4.1 Portée de tir . . . . . .

4.2 Placement incorrect du prefab sur le marqueur . . . . . .

4.3 Matériau des plans trop visible . . . . . .

4.4 Scripts mal placés dans la hiérarchie . . . . . .

4.5 Interaction tour–ennemis non détectée . . . . . .

5 Utilisation de l'Application Finale

5.1 Installation et Lancement . . . . . .

5.1.1 Étape 1 : Scanner l'environnement . . . . . .

5.1.2 Étape 2 : Placer la la zone de jeu . . . . . .

5.1.3 Étape 3 : Placer la tourelle . . . . . .

6 Architecture du Code

6.1 Séparation des Responsabilités . . . . . .

6.2 Paramètres Exposés . . . . . .

6.3 Gestion des Events . . . . . .

6.3.1 Pattern Observer . . . . . .

7 Améliorations Futures

7.1 Gameplay . . . . . .

7.1.1 Ennemis mobiles . . . . . .
7.1.2 Système de vagues . . . . . .
7.1.3 Types d'ennemis variés . . . . . .
7.1.4 Tourelles multiples . . . . . .
7.2 Interface Utilisateur . . . . . .
7.2.1 UI Canvas en AR . . . . . .
7.2.2 Boutons de contrôle . . . . . .
7.2.3 Feedback audio . . . . . .

8 Conclusion

8.1 Bilan du Projet . . . . . .

1 Introduction

Ce projet consiste en une application de réalité augmentée (AR) développée avec Unity. Nous avons choisi de créer un début de jeu de Tower Defense (i.e. des tourelles défendent une position contre des ennemis qui apparaissent dans l'environnement réel capturé par la caméra du smartphone).

2 Création des éléments de jeu

2.1 La tourelle (Tower)

Composition :

- Modèle importé depuis rigmodels.com
- Script `TowerController.cs` avec :
  - Système de détection d'ennemis dans un rayon défini
  - Cadence de tir configurable
  - LineRenderer pour l'effet visuel de tir
  - Système de logs pour debug

Exemple de paramètres :

```
public float range = 0.05f;       // 5cm en metres reels
public float damage = 1f;          // Degats par tir
public float fireRate = 1f;        // 1 tir par seconde
public bool showLogs = false;      // Debug optionnel
```

Listing 1 – Paramètres de la tourelle

2.2 Les ennemis (Enemies)

Structure :

- Prefab d'une sphère avec tag "Enemy" (pour la détection). On pourrait importer par la suite un objet 3D, mais nous sommes restés sur une sphère car cela permet de ne pas se soucier de la rotation ou des animations de l'objet en lui-même.
- Script `EnemyController.cs` gérant :
  - Points de vie (health)

  - Méthode `TakeDamage`
  - Destruction automatique quand la vie atteint 0
  - Barre de vie visuelle (Canvas en World Space)
- Spawn sur une position définie

2.3 Le gestionnaire de jeu (GameManager)

Le `GameManager.cs` centralise :

- Compteur d'ennemis tués
- Logs de progression du jeu
- Logique globale et état du jeu

À noter que pour l'instant les ennemis tournent en boucle (la carte a d'ailleurs été conçue ainsi), par la suite une carte pourrait être conçue pour que les ennemis atteignent un point final.

3 Techniques et Choix de Conception

3.1 Système de Coordonnées AR

Les distances en AR sont exprimées en **mètres réels**, différentes des unités Unity classiques où 1 unité peut représenter n'importe quelle distance selon l'échelle du jeu. Il y a donc eu un certain temps consacré à tester itérativement, compiler et relancer plusieurs fois l'application pour trouver les bonnes proportions.

3.2 Détection et Ciblage des Ennemis

Nous avons utilisé la méthode `GameObject.FindGameObjectsWithTag("Enemy")` pour identifier les ennemis (grandes sphères rouges sur l'image).
Nous avons utilisé un LineRenderer pour visualiser les tirs.

Figure 1 – Visualisation des tirs avec LineRenderer

3.3 Mouvement des ennemis

Nous avons choisi d'utiliser des "waypoints" pour tracer un chemin que les ennemis doivent suivre. Ces points sont d'ailleurs visibles sur la figure 3 (petites sphères rouges, différentes des ennemis qui sont des grandes sphères avec barre de vie). Dans une future version du jeu, ces points seront invisibles.

4 Problèmes Rencontrés et Solutions

4.1 Portée de tir

Problème. La tourelle tirait sur des ennemis visuellement très éloignés, rendant le jeu sans difficulté.

Cause. Provient directement du souci expliqué précédemment (échelles Unity et AR différentes). Valeur initiale `range = 5f` adaptée aux jeux 3D classiques mais représentant 5 mètres en réalité augmentée (distance excessive).

D'autre part, nous n'avions aucun retour visuel de la portée de tir. Nous avons donc ajouté un disque vert transparent autour de la tour, donnant un feedback sur la portée de tir.

4.2 Placement incorrect du prefab sur le marqueur

Problème. Le modèle 3D apparaissait mal orienté ou mal échelonné sur le marqueur.
Solution. Ajustement manuel de la rotation du plan par la création d'un objet parent assigné au `ARTrackedImageManager`, permettant une manipulation libre des coordonnées du sous-plan compensant celles du parent.

4.3 Matériau des plans trop visible

Problème. Les plans détectés étaient trop opaques et gênaient la visibilité.
Cause. Le matériau utilisé par le `ARPlaneManager` était opaque.
Solution. Passage du matériau en mode *Transparent* et réduction de l'alpha dans la couleur de base.

4.4 Scripts mal placés dans la hiérarchie

Problème. Certains scripts ne fonctionnaient pas comme prévu.
Cause. Les scripts n'étaient pas attachés aux bons objets (ex. `EnemySpawner` sur un objet hors du plateau, le script crash dès le lancement de l'application car le plateau n'est pas encore apparu).
Solution. Réorganisation : `BoardPlacement` sur l'objet AR principal, `EnemySpawner` et `GameManager` sur le plateau, `TowerController` sur l'objet issu du marqueur. Ainsi, tous les scripts se lancent au bon moment.

4.5 Interaction tour–ennemis non détectée

Problème. La tour ne détruisait pas les ennemis.

Cause. Les ennemis n'avaient pas le tag `Enemy`, ou la portée de la tour était trop faible.

Solution. Ajout du tag `Enemy` sur le prefab, ajustement du rayon d'action et vérification des distances.

5 Utilisation de l'Application Finale

5.1 Installation et Lancement

Ouvrir le projet depuis Github, connecter le smartphone Android et lancer la compilation. Suivre les étapes :

5.1.1 Étape 1 : Scanner l'environnement

Figure 2 – Grille AR transparente

1. Orienter le téléphone vers le sol ou une table
2. Bouger lentement de gauche à droite
3. Attendre l'apparition de la grille AR blanche (transparente), visible sur l'image à gauche.
4. Continuer jusqu'à avoir une zone stable et assez grande.

5.1.2 Étape 2 : Placer la la zone de jeu

1. Toucher l'écran sur un plan détecté
2. La map devrait apparaître instantanément

5.1.3 Étape 3 : Placer la tourelle

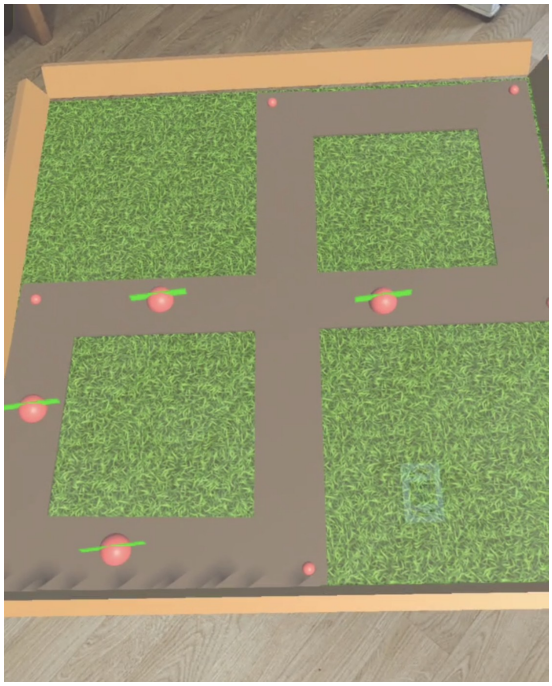

Figure 3 – Grille AR transparente

- Les ennemis spawn automatiquement sur la map
- Utiliser le marker (nous avons utilisé une carte de jeu "Roi", elle est un peu visible sur l'image, en bas à droite sous l'herbe.
- Approcher le téléphone de la carte si la tourelle ne spawn pas
- La tourelle tirera automatiquement dans sa portée, visible par un cercle vert (cf. figure 1)

6 Architecture du Code

6.1 Séparation des Responsabilités

Chaque script a une responsabilité unique et bien définie:

Script	Responsabilité
PlacementController	Interface utilisateur (touch), Interaction AR, Instanciation
TowerController	Logique de combat, Détection ennemis, Effets visuels
EnemyController	Gestion de vie, Réception dégâts, Comportement ennemi
GameManager	État global, Statistiques, Coordination

Table 1 – Répartition des responsabilités

6.2 Paramètres Exposés

```
public class TowerController : MonoBehaviour {
    public float range = 0.05f;       // Modifiable sans recompiler
    public float fireRate = 1f;        // Ajustable en temps reel
    public bool showLogs = false;      // Debug on off
```

```
}
```

Listing 2 – Exemple de paramètres publics

6.3 Gestion des Events

6.3.1 Pattern Observer

Utilisation de méthodes callback pour les événements :

```
// Quand un ennemi meurt
GameManager.Instance.OnEnemyKilled();
```

7 Améliorations Futures

7.1 Gameplay

7.1.1 Ennemis mobiles

- Déplacement vers un objectif à défendre
- Détection d'obstacles AR
- Animation de marche

7.1.2 Système de vagues

- Spawn progressif par vagues avec pauses entre les vagues
- Difficulté croissante, ennemi de type "boss"

7.1.3 Types d'ennemis variés

- Ennemis rapides / lents (faire varier la vitesse)
- Ennemis résistants / fragiles (faire varier les HP)
- Ennemis volants (imaginer éventuellement des parcours alternatifs, avec une carte contenant des fossés, murs...)

7.1.4 Tourelles multiples

- Utiliser les autres cartes (Dame, Valet, etc.) pour placer d'autres tourelles.

— Ressources gagnées en tuant des ennemis
— Différents types de tourelles, qu'on peut acheter avec les ressources
— Amélioration/upgrade des tourelles

7.2 Interface Utilisateur

7.2.1 UI Canvas en AR

— Affichage des stats en temps réel (argent, vague, santé)
— Interface en World Space attachée à l'AR

7.2.2 Boutons de contrôle

— Pause / Resume du jeu
— Restart après défaite
— Menu de sélection de tourelle
— Zoom et navigation caméra

7.2.3 Feedback audio

— Sons de tir différenciés par type
— Explosion à la destruction d'ennemis
— Musique d'ambiance
— Alertes sonores (vague, défaite)

8 Conclusion

8.1 Bilan du Projet

Ce projet a permis d'explorer les bases de la réalité augmentée avec Unity AR Foundation. Les principaux défis rencontrés ont été :

1. **Compréhension de l'échelle AR** : Adaptation des distances et proportions du jeu classique vers les mètres réels de l'AR
2. **Configuration mobile** : Gestion des spécificités Android, ARCore, et permissions

3. **Debug sur appareil** : Difficulté à débugger sans connection directe, utilisation intensive de logs et tests itératifs

Fin du rapport